MADHUBEN & BHANUBHAI PATEL INSTITUTE OF TECHNOLOGY
(THE CHARUTAR VIDYA MANDAL (C V M) UNIVERSITY)
DEPARTMENT OF COMPUTER ENGINEERING

LINKED LIST

Singly Linked List | Data Structures | Programming Language: C

Academic Year: 2025-2026 | **Subject:** Data Structure (102040304) | **Class:** 3CE1-B

Submitted by (Group 7): Harshita Mohapatra (12502040701039), Harshkumar Pande (12502040701040)

Submitted to: Dr. Maulika Patel

1. Introduction

A **Singly Linked List** is a linear data structure in which elements are stored in individual nodes connected through pointers. Each node contains data and a pointer to the next node. Unlike arrays, linked lists can grow or shrink dynamically during program execution.

2. Objectives

- To understand the concept and structure of a singly linked list.
- To learn insertion, traversal, searching, and deletion operations.
- To understand pointers and dynamic memory allocation in C.
- To apply linked list concepts to a practical problem.

3. Structure and Working

Each node consists of two parts: **data** and a **next pointer**. The `head` pointer stores the address of the first node, while the last node points to `NULL`. Nodes are created dynamically using `malloc()` and their memory is released using `free()` when they are deleted.

Basic Structure:



The main operations include:

- **Insertion:** Adds a new node to the linked list.
- **Traversal:** Visits and displays each node.
- **Searching:** Finds a node containing the required data.
- **Deletion:** Removes a node by adjusting pointers.
- **Counting:** Calculates the total number of nodes.

4. Project Application

The concepts of singly linked lists are implemented in the **Campus Event Registration and Participant Management System** using C. Each node stores a participant's registration ID, name, event name, category, and a pointer to the next participant. The system supports registration, displaying records, searching, updating details, cancelling registrations, and counting participants.

5. Advantages

- Dynamic memory allocation.
- Flexible insertion and deletion of nodes.
- No need for contiguous memory locations.
- Efficient memory utilization when nodes are managed properly.

6. Conclusion

This project demonstrates the practical application of singly linked lists in managing participant records. It strengthens our understanding of pointers, dynamic memory allocation, and fundamental linked list operations while developing programming and problem-solving skills in C.